

SESSION 10

EXPERIMENT 19 — VIRTUAL FILE SYSTEM (VFS)

Aim

To study the **Virtual File System (VFS) architecture** in Linux and demonstrate common filesystem system calls such as `open()`, `read()`, `write()`, `close()`, and `stat()`.

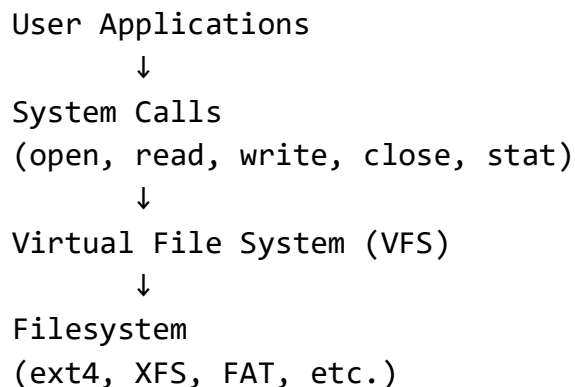
Theory

The **Virtual File System (VFS)** is a software layer in the Linux kernel that provides a common interface for accessing different filesystems.

It allows applications to use the same system calls regardless of whether the underlying filesystem is ext4, XFS, FAT, or another supported filesystem.

VFS Architecture

A simplified architecture is:



↓
Block Device / Storage

Major VFS Components

Superblock: Contains information about a mounted filesystem.

Inode: Contains metadata about a file.

Dentry: Represents a directory entry and connects a filename to an inode.

File Object: Represents an open instance of a file and contains information such as the file offset and access mode.

Common System Calls

System Call	Purpose
<code>open()</code>	Opens or creates a file
<code>read()</code>	Reads data
<code>write()</code>	Writes data
<code>close()</code>	Closes a file
<code>stat()</code>	Retrieves file metadata

Algorithm

1. Create or open a file using `open()`.
2. Obtain its file descriptor.
3. Write data using `write()`.
4. Close the file.
5. Reopen the file in read-only mode.
6. Read the contents using `read()`.
7. Display the contents.
8. Use `stat()` to obtain file metadata.
9. Display the file size and inode number.

10. Close the file.

C Program

```
C vfs_demo.c > main()
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <fcntl.h>
4  #include <unistd.h>
5  #include <sys/stat.h>
6
7  int main() {
8
9      int fd;
10     char buffer[200];
11
12     const char *filename = "vfs_file.txt";
13     const char *data =
14         "This file demonstrates Linux VFS system calls.\n";
15
16     // Open/Create file
17     fd = open(filename, O_CREAT | O_WRONLY | O_TRUNC, 0644);
18
19     if (fd == -1) {
20         perror("open");
21         return 1;
22     }
23
24     printf("File opened successfully.\n");
25     printf("File Descriptor: %d\n", fd);
26
27     // Write data
28     if (write(fd, data, 50) == -1) {
29         perror("write");
30         close(fd);
31         return 1;
32     }
33
34     printf("Data written successfully.\n");
35 }
```

```

29     perror("write");
30     close(fd);
31     return 1;
32 }
33
34 printf("Data written successfully.\n");
35
36 close(fd);
37
38 // Reopen file for reading
39 fd = open(filename, O_RDONLY);
40
41 if (fd == -1) {
42     perror("open");
43     return 1;
44 }
45
46 int bytes = read(fd, buffer, sizeof(buffer) - 1);
47
48 if (bytes == -1) {
49     perror("read");
50     close(fd);
51     return 1;
52 }
53
54 buffer[bytes] = '\0';
55
56 printf("\nFile contents:\n");
57 printf("%s", buffer);
58
59 close(fd);
60
61 // Get file metadata
62 struct stat info;
63

```

```
54     buffer[bytes] = '\0';
55
56     printf("\nFile contents:\n");
57     printf("%s", buffer);
58
59     close(fd);
60
61     // Get file metadata
62     struct stat info;
63
64     if (stat(filename, &info) == -1) {
65         perror("stat");
66         return 1;
67     }
68
69     printf("\nFile Metadata:\n");
70     printf("Inode Number: %lu\n",
71         (unsigned long)info.st_ino);
72
73     printf("File Size: %ld bytes\n",
74         (long)info.st_size);
75
76     printf("Permissions: %o\n",
77         info.st_mode & 0777);
78
79     return 0;
80 }
```

OUTPUT:

```

chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ gcc vfs_demo.c -o vfs_demo
vfs_demo.c: In function 'main':
vfs_demo.c:28:9: warning: 'write' reading 50 bytes from a region of size 48 [-Wstringop-overread]
   28 |         if (write(fd, data, 50) == -1) {
       |             ^~~~~~
In file included from vfs_demo.c:4:
/usr/include/unistd.h:378:16: note: in a call to function 'write' declared with attribute 'access (read_only, 2, 3)'
   378 | extern ssize_t write (int __fd, const void *__buf, size_t __n) __wur
       |             ^~~~~
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ ./vfs_demo
File opened successfully.
File Descriptor: 3
Data written successfully.

File contents:
This file demonstrates Linux VFS system calls.

File Metadata:
Inode Number: 7599824371968401
File Size: 50 bytes
Permissions: 777
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$

```

Result

Thus, the Linux Virtual File System was studied and successfully demonstrated using the `open()`, `read()`, `write()`, `close()`, and `stat()` system calls. File data and metadata were successfully accessed through the VFS interface.

EXPERIMENT 20 — MEMORY-MAPPED FILES

Aim

To study **memory-mapped files** and demonstrate mapping a file into the process address space using the `mmap()` system call.

Theory

A **memory-mapped file** is a file that is mapped directly into a process's virtual address space.

Instead of repeatedly using `read()` and `write()`, a program can access the mapped file like a region of memory.

The basic process is:

```
File
  ↓
mmap()
  ↓
Virtual Address Space
  ↓
Process accesses file as memory
```

`mmap()`

The `mmap()` system call maps a file or other object into the process's virtual memory.

Advantages

- Efficient file access.
- Useful for large files.
- Allows file contents to be accessed like an array in memory.
- Can reduce unnecessary copying.

Demand Paging

Memory-mapped files are commonly loaded using **demand paging**.

The complete file does not necessarily need to be loaded into RAM immediately. Pages can be brought into memory when they are accessed.

Page Fault

If the process accesses a mapped page that is not currently in physical memory, a page fault occurs and the operating system can load the required page.

Algorithm

1. Create/open a file.
2. Write initial data into the file.
3. Set the required file size.
4. Map the file into memory using `mmap()`.
5. Access the file contents through the mapped memory.
6. Modify the mapped data.
7. Synchronize changes using `msync()`.
8. Unmap the memory using `munmap()`.
9. Close the file.
10. Display the modified file contents.

C Program — Memory-Mapped File

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <string.h>

#define FILE_SIZE 100

int main() {

    int fd;
    char *mapped_memory;

    const char *filename = "mapped_file.txt";

    // Create/open file
    fd = open(filename, O_RDWR | O_CREAT | O_TRUNC, 0644);

    if (fd == -1) {
        perror("open");
        return 1;
    }

    // Set file size
    if (ftruncate(fd, FILE_SIZE) == -1) {
        perror("ftruncate");
        close(fd);
        return 1;
    }

    // Map file into memory
    mapped_memory = mmap(
```

```
        perror("ftruncate");
        close(fd);
        return 1;
    }

    // Map file into memory
    mapped_memory = mmap(
        NULL,
        FILE_SIZE,
        PROT_READ | PROT_WRITE,
        MAP_SHARED,
        fd,
        0
    );

    if (mapped_memory == MAP_FAILED) {
        perror("mmap");
        close(fd);
        return 1;
    }

    printf("File successfully mapped into memory.\n");

    // Write through mapped memory
    strcpy(mapped_memory,
        "Hello from a memory-mapped file!");

    printf("Data written through mapped memory.\n");

    // Synchronize changes with file
    if (msync(mapped_memory, FILE_SIZE, MS_SYNC) == -1) {
        perror("msync");
    }

    printf("Changes synchronized with the file.\n");
```

```

printf("Data written through mapped memory.\n");

// Synchronize changes with file
if (msync(mapped_memory, FILE_SIZE, MS_SYNC) == -1) {
    perror("msync");
}

printf("Changes synchronized with the file.\n");

// Display mapped data
printf("Mapped data: %s\n", mapped_memory);

// Unmap memory
munmap(mapped_memory, FILE_SIZE);

close(fd);

printf("Memory unmapped successfully.\n");

return 0;
}

```

OUTPUT:

```

chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ gcc mmap_demo.c -o mmap_demo
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ ./mmap_demo
File successfully mapped into memory.
Data written through mapped memory.
Changes synchronized with the file.
Mapped data: Hello from a memory-mapped file!
Memory unmapped successfully.
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$

```

Result

Thus, memory-mapped file functionality was successfully demonstrated using the `mmap()` system call. The file was mapped into the process address space, modified through memory, synchronized using `msync()`, and successfully unmapped using `munmap()`.

